Production-Ready Application Deployment

Project Overview

This project demonstrates a production-ready DevOps workflow for deploying a containerized application to AWS using Infrastructure as Code (Terraform), Docker, GitHub Actions CI/CD, Amazon ECS (Fargate), and CloudWatch monitoring.

The solution was designed with a strong focus on:

- Automation
- Scalability
- Reusability
- Maintainability
- Real-world DevOps practices

The deployment process is fully automated from code commit to production deployment.

Objectives

The primary objectives of this project are:

- Provision AWS infrastructure using Terraform
- Containerize the application using Docker
- Automate deployment using GitHub Actions
- Deploy the application to AWS ECS Fargate
- Implement monitoring and logging using AWS CloudWatch
- Maintain a modular and production-style DevOps workflow

Solution Architecture

Architecture Flow

Developer → GitHub Repository → GitHub Actions CI/CD →

Docker Build → Docker Hub →

AWS ECS (Fargate) →

CloudWatch Monitoring & Log

Architecture Components

Component	Purpose
GitHub	Source code management
GitHub Actions	CI/CD automation
Docker	Application containerization
Docker Hub	Docker image registry
Terraform	Infrastructure provisioning
AWS ECS Fargate	Container orchestration
AWS CloudWatch	Monitoring and logging
IAM Roles	ECS execution permissions
VPC/Subnets	Network infrastructure

Technologies Used

Cloud Platform

- AWS

Infrastructure as Code

- Terraform

CI/CD

- GitHub Actions

Containerization

- Docker

Container Orchestration

- Amazon ECS Fargate

Monitoring

- AWS CloudWatch

Version Control

- Git & GitHub

Project Structure

```
devops-project/
|
├── app/
|   ├── app.py
|   ├── requirements.txt
|   └── Dockerfile
|
├── terraform/
|   ├── provider.tf
|   ├── vpc.tf
|   ├── ecs.tf
|   ├── iam.tf
|   ├── variables.tf
|   └── outputs.tf
|
├── .github/
|   └── workflows/
|       └── deploy.yml
|
├── README.md
└── .gitignore
```

Docker Implementation

Dockerfile

The application was containerized using Docker to ensure:

- Consistent deployments
- Portability
- Scalability
- Environment isolation

Docker Workflow

1. Build Docker image
2. Tag image
3. Push image to Docker Hub
4. ECS pulls latest image automatically

☁️ **AWS Infrastructure Deployment**

Infrastructure provisioning was implemented using Terraform.

AWS Resources Provisioned

Networking

- VPC
- Public Subnets
- Internet Gateway
- Route Tables

Security

- Security Groups
- IAM Roles

Compute

- ECS Cluster
- ECS Task Definition
- ECS Service

Monitoring

- CloudWatch Log Groups

Infrastructure as Code (Terraform)

Terraform was used to automate AWS infrastructure provisioning.

Benefits

- Reusable infrastructure
- Version-controlled infrastructure
- Repeatable deployments
- Easy scalability

Terraform Commands Used

Initialize Terraform

terraform init

Validate Configuration

terraform validate

Preview Infrastructure Changes

terraform plan

Deploy Infrastructure

terraform apply

CI/CD Pipeline Implementation

GitHub Actions was used to automate the deployment workflow.

CI/CD Workflow

The deployment pipeline automatically executes whenever code is pushed to the main branch.

CI/CD Steps

1. Source Code Checkout

GitHub Actions checks out the repository code.

2. Configure AWS Credentials

AWS credentials are securely loaded using GitHub Secrets.

3. Docker Authentication

Pipeline authenticates with Docker Hub.

4. Build Docker Image

Application image is built automatically.

5. Push Docker Image

Docker image is pushed to Docker Hub.

6. Deploy to ECS

Amazon ECS service is updated automatically.

🔐 GitHub Secrets Configuration

Sensitive credentials were securely stored using GitHub Repository Secrets.

Configured Secrets

Secret Name	Purpose
AWS_ACCESS_KEY	AWS Authentication
AWS_SECRET_KEY	AWS Authentication
DOCKER_USERNAME	Docker Hub Login
DOCKER_PASSWORD	Docker Hub Login

🚀 ECS Deployment

Amazon ECS Fargate was selected because:

- Serverless container management
- No EC2 management required
- Easy scalability
- Simplified deployment workflow

ECS Components Used

- ECS Cluster
- ECS Service
- ECS Task Definition
- Fargate Launch Type

Monitoring & Logging

AWS CloudWatch was configured for:

- Container logs
- Application monitoring
- ECS task monitoring
- Deployment troubleshooting

CloudWatch Features Implemented

- ECS task logs
- Container stdout/stderr logs
- Centralized logging

🔒 **Security Considerations**

The following security best practices were implemented:

- IAM Roles for ECS task execution
- GitHub Secrets for sensitive credentials
- No hardcoded credentials
- Security Group restrictions

- Infrastructure managed through Terraform

Design Decisions

Why ECS Fargate?

- Simplifies container management
- Reduces infrastructure overhead
- Production-ready managed service

Why Terraform?

- Industry-standard IaC tool
- Reusable and modular infrastructure
- Easier environment replication

Why GitHub Actions?

- Native GitHub integration
- Simple CI/CD automation
- Fast pipeline setup

Why Docker?

- Consistent runtime environment
- Portable deployments
- Simplified dependency management

⚠️ **Limitations**

Current implementation limitations include:

- No Application Load Balancer (ALB)
- No Auto Scaling
- Single environment deployment
- Basic monitoring only

Future Improvements

Potential enhancements:

- Add Application Load Balancer (ALB)
- Implement Auto Scaling
- Add HTTPS with ACM
- Use AWS ECR instead of Docker Hub
- Implement Blue/Green deployment
- Add Prometheus & Grafana monitoring
- Add Kubernetes (EKS) deployment option

Testing & Validation

The following validations were performed:

- Terraform validation
- Docker image build test
- ECS deployment verification
- CloudWatch log verification
- GitHub Actions pipeline execution

Deployment Workflow Summary

Code Push →

GitHub Actions Trigger →

Docker Build →

Docker Push →

ECS Deployment →

CloudWatch Monitoring

Assumptions Made

- AWS account already configured
- Docker Hub account available

- GitHub repository configured
- Terraform installed locally
- AWS CLI configured locally

Author

Taiwo Peter Olatunji
DevOps Engineer Practical Challenge Submission

Repository

GitHub Repository Link:

https://github.com/Taiwo-Peter2023/devops-project

Conclusion

This project successfully demonstrates a production-style DevOps deployment pipeline using modern DevOps tools and AWS cloud services.

The implementation provides:

- Infrastructure automation
- CI/CD automation
- Containerized deployment
- Cloud monitoring
- Production-ready deployment practices